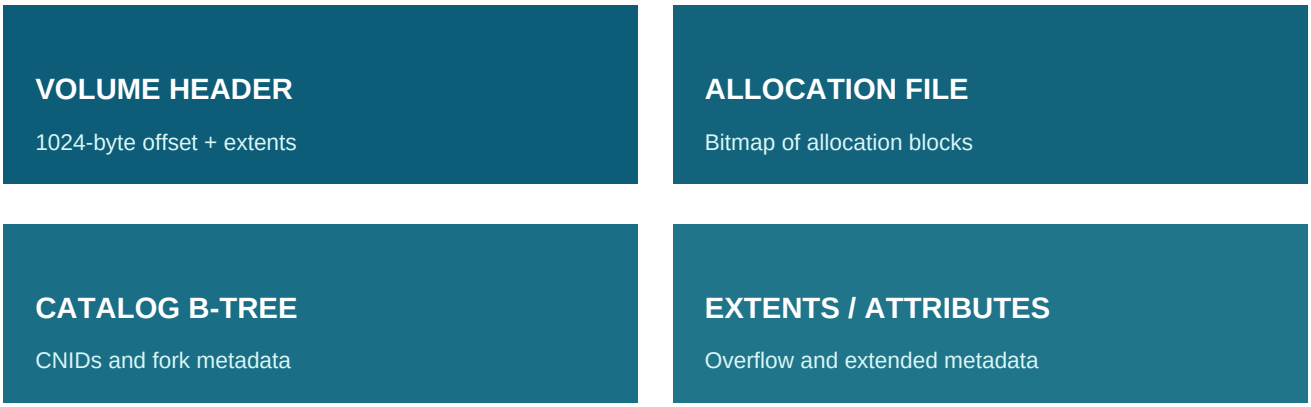
HFS Plus - Mac OS Extended format

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

Apple’s 1998 successor to HFS expanded allocation addressing and Unicode names while retaining B-tree catalogs and dual file forks.

This guide documents the original format’s technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

HFS Plus - Mac OS Extended format

HISTORICAL CHRONOLOGY

HFS Plus arrived with Mac OS 8.1 in 1998 as Mac OS Extended. Apple retained the familiar Macintosh catalog, extent and fork concepts while replacing limitations of original HFS. Apple's TN1150 describes smaller average allocation-block waste on larger drives as a central motivation and specifies differences in filenames and metadata representation.

During Mac OS X's lifetime, HFS Plus acquired optional journalling, hard-link handling, extensive permissions/attributes and the case-sensitive HFSX variant. APFS later replaced it as the preferred modern Apple filesystem on many devices. HFS Plus and HFSX must be identified as distinct modes and not treated as identical to original HFS.

HFS Plus - Mac OS Extended format

ORIGINAL ON-DISK ARCHITECTURE

The HFS Plus Volume Header is located 1024 bytes from the start of the volume. The format organises its special metadata as files: allocation, catalog, extents overflow, attributes and startup files, alongside user file forks. The catalog, extents and attributes structures use B-trees with node sizes and key formats specified by the format.

HFS Plus identifies allocation blocks with 32-bit numbers, permitting many more allocation units than original HFS. Multi-byte fields are big-endian. File and folder identifiers are CNIDs. Fork data descriptors include logical size, clump size, allocated blocks and extents, while overflow records extend the on-inode extent capacity.

HFS Plus - Mac OS Extended format

ALLOCATION AND DIRECTORIES

The allocation file records used blocks, while the catalog B-tree maps parent IDs and Unicode names to file and directory records. HFS Plus names use a 16-bit length and up to 255 UTF-16 code units with a version-specific canonical decomposition/collation scheme. A normal Unicode casefold or current host locale is not equivalent to Apple’s defined comparison algorithm.

A file retains both data and resource forks. Attributes B-trees can contain additional named metadata. HFSX adds support for case-sensitive keys and specific comparison rules. Hard links and symbolic links in Mac OS X use defined catalog and metadata-directory conventions rather than raw Unix inodes.

HFS Plus - Mac OS Extended format

INTEGRITY AND RECOVERY

The optional journal records metadata updates and permits quicker repair after an unsafe shutdown. Apple’s technical note makes clear that the journal is for volume structures, not a guarantee of full user-data persistence. Journal replay must preserve transaction order and issue flushes before advancing the journal start pointer.

A full checker validates catalog and extents tree nodes, CNIDs, link counts, allocation ownership, journal state and fork extents. Duplicate or orphaned metadata-directory entries and mismatched attribute records need special handling. Journal existence does not eliminate the possibility of damaged metadata or lost data during storage faults.

HFS Plus - Mac OS Extended format

DESIGN RATIONALE AND LIMITATIONS

HFS Plus solves several practical HFS limits while retaining compatibility with Macintosh dual-fork and Finder conventions. Its Unicode rules are historically specific and can differ from modern Unicode normalization. Integer timestamps use a 1904 epoch and have representational limits; time conversion must not assume Unix epochs.

It is a mutable B-tree format, not intrinsically copy-on-write. The case-sensitive HFSX variant and optional journal require explicit detection. APFS snapshots, encryption domains and object maps cannot be projected backward onto HFS Plus merely because both are Apple-designed.

HFS Plus - Mac OS Extended format

INFILTRATOROS INTEGRATION BOUNDARY

HFSPLUS.so must respect Apple’s HFS Plus Volume Format specification including big-endian structures, catalog keys, forks and allocation bitmap. Any HFSX support must be specified separately; shared implementation code does not make every HFS Plus volume case-sensitive.

The exact Apple TN1150 document is the primary technical authority. Native ‘.so‘, optional Linux ‘.ko‘ and InfiltratorOS packaging are separate from the on-media standard and require independent qualification.

HFS Plus - Mac OS Extended format

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. Apple TN1150, HFS Plus Volume Format (on-disk normative description)

https://developer.apple.com/library/archive/technotes/tn/tn1150.html

2. Contemporary introduction to original Macintosh HFS

https://preserve.mactech.com/articles/mactech/Vol.02/02.01/HFS/index.html

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

HFSPlus / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Locate the HFS Plus volume header and verify its signature, version, allocation block size, file counts and special-file fork descriptors.

2. Read the allocation file to discover free blocks; the catalog, extents overflow, attributes and startup files each have different roles.

3. Traverse the catalog B-tree for folders, files and thread records. File records hold both data-fork and resource-fork descriptions.

4. Resolve the first extents of each fork in its catalog record, then use the extents overflow B-tree for further extents.

5. Read the attributes B-tree for supported extended-attribute storage. Its records must not be confused with ordinary catalog entries.

6. Interpret names using HFS Plus Unicode normalisation and comparison rules. A host OS’s default Unicode casefolding is not guaranteed to match the disk format.

7. Distinguish journaled HFS Plus and case-sensitive variants by their actual on-media attributes, not a generic ’Mac OS Extended’ label.

8. Use the alternate volume header for corroboration, but verify its generation and fields rather than assuming it is a full snapshot of the filesystem.

HFSPlus / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Verify catalog and extent B-tree headers, node records, ordering and consistency between file records and thread records.

2. Cross-check allocation bitmap state with all fork extents, including resource forks and special metadata files.

3. A file with an empty data fork may still have important resource-fork contents, extended attributes and Finder metadata.

4. If journaling is enabled, assess the journal’s transaction state before modifying allocation or catalog metadata.

5. Preserve the disk’s name comparison rules. Renaming or creating a file with host-normalised characters can create collisions invisible to an incompatible reader.

6. Validate extent lengths and allocation-block bounds before translating a logical file position to a physical address.

7. Classic HFS and HFS Plus use different allocation and catalog layouts; a conversion history does not make their on-disk data interchangeable.

8. Keep format-level verification separate from macOS features layered above it, such as application metadata conventions or boot system policies.